

PRODUCTION READINESS & OPERATIONAL CONCERNS

LogLens

Operating the Platform in Production

How LogLens would run as a production service — service levels, failure modes, observability, rollout, cost control, and security — with what is addressed by the current design and what is deliberately deferred, and why.

Author & Architect: Dragan Solarov

Repository: github.com/solar-mkd/ai-log-intelligence-platform

Purpose & honesty of scope. LogLens is a reference implementation built to prove an architecture, not a deployed production service. This document sets out the operational concerns a production deployment must address, distinguishes clearly between what the current design *already provides*, what is *partially in place*, and what is *deliberately deferred* — and gives the reasoning for each deferral. The intent is to demonstrate that these concerns are understood and designed-around, not overlooked.

Operating Model & Reading Guide

A platform that ingests logs to diagnose other systems is itself infrastructure: when it fails, teams lose visibility precisely when they need it most. Its operational design therefore matters as much as its data design.

This document covers six concerns: service levels (SLA/SLO), failure-mode handling, observability, rollout strategy, cost control, and security & governance. Each section marks its items:

DESIGNED-IN already provided by the current architecture **PARTIAL** foundations present, completion deferred **DEFERRED** understood, not yet built — with rationale

Why deferral is a legitimate architectural position. A reference implementation should prove its core thesis before elaborating operational machinery. Building a full SLO/alerting/rollout stack before the architecture is validated would be premature optimization. The discipline is to *know* what production requires, *design so it can be added without rework*, and defer the build deliberately — which is what the layer isolation, stateless steps, and observability hooks already in the design enable.

1. Service Levels (SLA / SLO)

LogLens has two operationally distinct surfaces with very different service-level profiles. Treating them as one would be a mistake; separating them is the first design decision.

The ingest/processing pipeline (write path)

A batch/streaming pipeline whose service level is about **freshness** and **completeness**, not request latency.

Proposed SLOs:

SLO	Target (illustrative)	Rationale
Data freshness	95% of entries queryable within 15 min of arrival	Diagnostics need recent data; minutes, not seconds, is acceptable for log analysis
Completeness	99.9% of landed entries reach gold	Idempotency + per-file commits make loss rare; the 0.1% covers unparseable/failed files
Pipeline success	99% of source runs complete without manual intervention	Per-source isolation means one bad source doesn't fail others

DESIGNED-IN The architecture already supports these: hash idempotency guarantees re-runs are safe (so a missed window is recoverable by re-running), per-file commits bound data loss to at most one in-flight file, and per-source isolation contains failures.

The query/RAG surface (read path)

An interactive surface where **latency** matters. Here the honest constraint is the LLM:

SLO	Target (illustrative)	Note
Retrieval latency	p95 < 500 ms (vector + filter)	pgvector + HNSW makes retrieval itself fast; this is achievable now
End-to-end answer	p95 < 10 s (warm model)	Dominated by LLM generation, not retrieval
Availability	99.5% (read path)	Degrades gracefully: retrieval works even if the LLM is down (see §2)

An honest operational reality. The dominant latency factor is LLM inference, not the data platform. A cold model load can take far longer than a warm call. A production deployment would therefore keep the model warm (a resident inference service rather than on-demand load), and right-size the model to the latency target — a smaller model for snappy interactive use, a larger one for depth. This is a capacity decision, and the swappable-model design (the LLM is an interface, not a hard dependency) already makes it a configuration choice rather than a rebuild.

Targets are illustrative starting points for an SLO conversation with stakeholders, not commitments — the right numbers come from the actual operational context and user expectations.

2. Failure-Mode Handling

The pipeline's structure was chosen partly *for* its failure behaviour. The table maps the principal failure modes to how the current design responds and what production would add.

Failure	Current behaviour	Production addition
A single bad file (corrupt, unparseable)	Marked <i>failed</i> in the control table, skipped; the run continues. Per-file transaction rolls back only that file.	Alert on failed-file rate; dead-letter queue for inspection/replay
Crash mid-run	Committed files stay done; the run row is left <i>in_progress</i> at its last counts — diagnosable. Re-running is safe (idempotent).	Watchdog to detect stale <i>in_progress</i> runs and auto-retry/alert
Duplicate / re-delivered data	Entry-hash idempotency: re-ingested entries are no-ops.	— (already robust)
One source misconfigured/failing	Per-source isolation: its failure is logged; other sources proceed. Orchestrator returns non-zero but completes the rest.	Per-source health status surfaced to a dashboard
Missing PII key	Fail-closed: pipeline refuses to run rather than write unprotected PII.	Pre-flight config validation in CI / on deploy
LLM unavailable	Retrieval still works; only generation fails.	Graceful degradation: return retrieved evidence with a "LLM unavailable" notice rather than erroring
Database unavailable	Steps fail cleanly and are re-runnable; no partial-state corruption (transactional boundaries).	Connection retry/backoff; read replica for the query path
Embedding model change	Vectors are model-pinned in a separate table — old and new coexist without ambiguity.	Background re-embedding job with cutover

DESIGNED-IN The first column is not aspirational — idempotency, per-file commits, per-source isolation, fail-closed PII, transactional boundaries, and model-pinned vectors are all implemented. **DEFERRED** The third column (alerting, dead-letter queue, watchdog, graceful LLM degradation) is understood and deferred; the design's isolation makes each additive.

Design principle on display. Most of these failures are *contained* rather than *prevented* — a bad file, a crash, a down dependency cannot corrupt state or lose committed data, and recovery is "re-run." Designing for containment and safe re-run, rather than trying to prevent every failure, is the architecturally mature stance for a data pipeline.

3. Observability Depth

Observability is layered: a production system needs to answer "is it healthy?", "is the data right?", and "why is it slow/failing?" across the pipeline and the query surface.

What exists

PARTIAL The pipeline already records per-run observability in a `bronze_runs` table — status, files processed, entries landed, duration, host — updated after every file, so progress is visible live and a crashed run is diagnosable. Failures are logged per source. This is the foundation of operational telemetry.

What production needs (the three pillars)

Pillar	For LogLens specifically	Status
Metrics	Throughput (entries/min), freshness lag, failed-file rate, queue depth, retrieval p95, LLM latency, embedding backlog	DEFERRED emit to Prometheus/StatsD from the existing run-tracking hooks
Logs (of LogLens itself)	Structured logs per step with <code>run_id/source_id</code> correlation — the platform should be as observable as the systems it observes	PARTIAL failures logged; needs structured, correlated, levelled logging throughout
Traces	A query traced retrieval → prompt build → LLM call, to locate latency; an entry traced bronze → silver → gold for lineage	DEFERRED the GUID keys per entry/run already provide the correlation IDs tracing needs

Data-quality observability (often forgotten)

Beyond system health, a data platform must observe **data correctness**: parse-failure rate per source (a spike means a log format changed), unexpected severity distributions, sources that stopped sending (silent failure), and embedding coverage (segments without vectors). **DEFERRED** — but the schema makes these straightforward SQL checks to schedule.

The architect point. The system was built with observability *hooks* (the run table, GUID correlation keys, per-source status) rather than observability *plumbing*. That is the right order: design the seams that telemetry attaches to, defer the wiring to a specific stack (which is an environment decision) until deployment.

4. Rollout & Change Strategy

Three kinds of change carry different risk and need different rollout treatment.

Schema & pipeline-code changes

DESIGNED-IN Schema application is idempotent (IF NOT EXISTS throughout), so schema rollout is safe and repeatable. Steps are stateless and each layer re-runnable in isolation, so a fixed transform can re-process a layer without re-ingesting. **DEFERRED** Production would add: migrations with explicit up/down, a staging environment mirroring production, and CI gating on the test suite.

Parser changes / new log types

DESIGNED-IN A new log type is additive — one self-registering module, no change to existing parsers or the pipeline (proven by adding Apache). The parser version is recorded per row, so a parser change is traceable and its output distinguishable from prior versions. **DEFERRED** Re-processing under a new parser version as a controlled backfill job.

Embedding-model changes (the highest-risk change)

Changing the embedding model invalidates comparability between old and new vectors. **DESIGNED-IN** This was anticipated: embeddings are **pinned to the producing model in a separate table**, so a new model's vectors coexist with the old. **DEFERRED** The rollout pattern this enables: embed under the new model in parallel, evaluate retrieval quality (A/B) on the same queries, then cut the query path over — with rollback as simply pointing back at the old model's vectors.

Why this reads as architect-level. The riskiest production change (model migration) was made *safe by a design decision taken upfront* (ADR-012, model-pinned vectors), not patched later. Designing the rollback path before it is needed is the signal.

5. Cost Control

LogLens's cost profile is unusual and, by design, favourable — worth making explicit because it is a deliberate architectural outcome.

The dominant cost drivers

Driver	Behaviour	Lever
Storage	Medallion keeps data in three layers (raw + silver + gold) — more storage by design	Bronze archival/retention tiering (roadmap); raw is the largest and the most compressible/coldest
Embedding compute	One-off per segment; skipped if already embedded (idempotent)	Batch sizing; already avoids re-embedding unchanged segments
LLM inference	Per query; the most variable cost, and CPU/GPU-bound	Model right-sizing; the swappable-model design makes this a config choice
Database	HNSW index + vectors have a memory footprint that grows with corpus	Index tuning; partitioning/retention on gold

The structural cost advantage

DESIGNED-IN Because embeddings and the LLM run **locally**, there are **no per-token or per-API-call charges** — the cost is owned hardware, not metered cloud AI. For high or unpredictable query volumes this is dramatically cheaper and more predictable than a hosted-LLM RAG, and it removes per-call cost as a scaling worry. It also caps data-egress and third-party exposure at zero (logs never leave the environment), which is a cost *and* a governance benefit.

Cost as an architectural choice. "Local-first" was chosen primarily for privacy, but it doubles as cost control: it converts a variable, usage-metered AI bill into a fixed, capacity-planned hardware cost. Recognizing that one decision serves privacy, cost, and governance simultaneously is the kind of cross-cutting reasoning the role calls for.

DEFERRED Formal cost monitoring (storage growth trending, per-query compute attribution, retention-policy automation) is roadmap, enabled by the same observability hooks in §3.

6. Security & Governance

This is the area where LogLens has the most *built*, not merely designed — because handling logs means handling sensitive data from the outset.

What is implemented

- **DESIGNED-IN** **Per-field, per-source PII policy** — redaction and keyed-HMAC pseudonymisation, applied at silver ingestion, configurable per source (ADR-015).
- **DESIGNED-IN** **Fail-closed enforcement** — the pipeline refuses to run if an HMAC field lacks its key, so it cannot silently write unprotected PII.
- **DESIGNED-IN** **Secrets never committed** — connection strings and keys come from environment/local config only; real config and secrets are git-ignored, with a placeholder example checked in.
- **DESIGNED-IN** **Local-first data residency** — embeddings and LLM run on-machine; log data never leaves the environment, eliminating third-party data exposure.
- **DESIGNED-IN** **Full lineage** — every entry is traceable bronze → silver → gold via stable keys, and the raw entry is retained for audit (normalize-but-retain-raw).

What production would add

Concern	Production requirement	Status
Access control	AuthN/Z on the query surface; row/source-level authorization so users see only permitted sources	DEFERRED
Encryption	At rest (DB/volume) and in transit (TLS to DB and inference); the AES PII action for reversible fields	PARTIAL AES hook reserved; transport/at-rest are deployment config
Key management	The PII HMAC key in a secrets manager with rotation policy (noting rotation changes fingerprints — a documented trade-off)	DEFERRED
Audit of access	Who queried what, when — an audit log of the read path, not just data lineage	DEFERRED
Retention & right-to-erasure	Policy-driven retention; given pseudonymisation, erasure strategy for the keyed fields	DEFERRED

Governance posture. The design treats PII and lineage as *first-class, built-in* concerns rather than add-ons — fail-closed by default, raw retained for audit, secrets externalized. The deferred items (access control, KMS, access-audit, retention automation) are deployment-context decisions layered onto a base that was designed to be governed, not retrofitted for it.

Summary: Designed-In vs. Deferred

The recurring pattern across all six concerns is deliberate: the **architecture** already embeds the properties that make production operability *possible* — idempotency and safe re-run, per-file and per-source isolation, transactional containment, model-pinned vectors, observability hooks via run-tracking and GUID correlation keys, fail-closed governance, and local-first data residency. The **operational machinery** that builds on those properties — alerting, dashboards, watchdogs, CI/CD gating, KMS, access control, retention automation — is consciously deferred, because a reference implementation should validate its architecture before elaborating its operations.

The intent of this document is to show that these concerns are **understood, reasoned about, and designed for** — that the deferrals are choices with rationale, not blind spots. Knowing what production demands, and designing so it can be added without rework, is the architectural discipline; building it all upfront would be the opposite.

